

README / Setup Guide

Overview

This Enterprise Procurement and Inventory Management Platform is designed for a retail organization operating across India, with 150 stores, 12 warehouses, 2,000 suppliers, and 10,000 employees. The platform streamlines the entire procurement process, from identifying purchasing needs through supplier selection, purchase approval, delivery, inventory updates, invoice verification, and payment reconciliation. It aims to eliminate duplicate purchases, budget overruns, stock shortages, delayed payments, and limited audit visibility by providing a centralized, integrated system.

Key Features

- **Configurable Approval Workflows:** Automatically route purchase requisitions based on value and category, ensuring appropriate levels of approval.
- **Budget Controls:** Track and manage budget allocations, reservations, and spending across cost centers, categories, and financial years.
- **Supplier Onboarding and Quotation Management:** Facilitate supplier registration, quotation requests, and comparison to streamline supplier selection.
- **Role-Specific Dashboards:** Provide tailored views of pending approvals, budget utilization, purchase status, inventory levels, and more.

Prerequisites

- Java 21+
- Spring Boot
- PostgreSQL
- Node.js 16+
- npm 8+
- AWS account for deployment

Project Structure

- **backend/:** Contains the Spring Boot application, handling business logic, API endpoints, and database interactions.
- **frontend/:** React application with TypeScript, providing the user interface and interacting with the backend via REST APIs.
- **shared/:** Shared utilities and common code between frontend and backend.
- **docs/:** Documentation and project details.

Installation & Setup

Backend Setup

1. Clone the repository: `git clone <url>`
2. Navigate to the project: `cd project-name`
3. Create virtual environment: `python -m venv venv`
4. Activate: `source venv/bin/activate` (Linux/Mac) or `venv\Scripts\activate` (Windows)
5. Install dependencies: `pip install -r requirements.txt`
6. Configure PostgreSQL database: Set up a PostgreSQL instance and update the `application.properties` file with the database connection details.

Frontend Setup

1. Navigate to frontend: `cd frontend`
2. Install dependencies: `npm install`
3. Build/prepare assets: `npm run build`

Configuration

Required Environment Variables

- `SPRING_DATASOURCE_URL`: Database connection URL (e.g., `jdbc:postgresql://localhost:5432/procurement`)
- `SPRING_DATASOURCE_USERNAME`: Database username
- `SPRING_DATASOURCE_PASSWORD`: Database password
- `JWT_SECRET`: Secret key for JWT token generation
- `MAIL_USERNAME`: Email provider username for sending notifications
- `MAIL_PASSWORD`: Email provider password
- `AWS_ACCESS_KEY_ID`: AWS access key for object storage
- `AWS_SECRET_ACCESS_KEY`: AWS secret key for object storage

Running the Application

Starting the Backend

1. Ensure virtual environment is activated
2. Set environment variables: `export`
`SPRING_DATASOURCE_URL=jdbc:postgresql://localhost:5432/procurement`

3. Start server: `mvn spring-boot:run`
4. Verify: Backend is running on `http://localhost:8080`

Starting the Frontend

1. Navigate to frontend directory: `cd frontend`
2. Start dev server: `npm run dev`
3. Open browser: `http://localhost:3000` (or displayed URL)
4. Verify: UI loads and connects to backend

Verification Checklist

- ☐ Backend health check responds at `GET /api/health` with 200 OK
- ☐ Frontend loads without console errors
- ☐ Basic workflow completes end-to-end (e.g., create a purchase requisition and track its approval)
- ☐ API endpoints respond with expected data format

Troubleshooting

Backend Issues

- **Port already in use:** Change PORT env var or kill process on port 8080
- **Module not found:** Run `pip install -r requirements.txt` again
- **Database connection error:** Verify connection string in `application.properties`, check service is running
- **Authentication error:** Verify API keys/tokens in `.env` are correct and not expired

Frontend Issues

- **Blank page or won't load:** Check browser console for errors, ensure backend is running
- **API connection error:** Verify backend URL in config, check CORS settings
- **Module resolution error:** Delete `node_modules` and run `npm install` again
- **Build fails:** Ensure Node.js version matches requirements, clear cache: `npm cache clean --force`

Common Workflows

Create a Purchase Requisition

1. Navigate to the requisition creation page.
2. Enter product details, quantities, estimated prices, delivery dates, and justifications.

3. Submit the requisition for approval.
4. Verify the requisition appears in the list of submitted requisitions.

Approve a Purchase Requisition

1. Navigate to the approval dashboard.
2. Select a requisition to approve.
3. Review the details and approve or reject the requisition.
4. Verify the requisition status updates to "Approved" or "Rejected".

Deployment Notes

- Ensure environment variables are securely managed, especially database credentials and API keys.
- Configure AWS services for database, object storage, and notifications.
- Set up load balancers and auto-scaling for high availability.

Getting Help

- Check logs: Backend logs in console, frontend logs in browser DevTools
- Review error message and stack trace for clues
- Check environment variables are set correctly
- Refer to the API Documentation and User Guide for more details